

Indian Institute of Technology Indore

Science & Technology Council

IITISoC 2026 — Project Proposal

Selected Problem Statement (Preference 1):

Personalized Mental Health Digital Twin using AI

Domain: Artificial Intelligence & Machine Learning

Difficulty: Advanced

Team Size: 4–6

1. Team Details

1.1 Team Leader

Name ROHITH M S
Roll Number 250005038
Discord discord.com/users/1497592055764090964
GitHub github.com/rohithsuresh21
LinkedIn linkedin.com/in/rohith-m-s-964511378
Skills PyTorch, Neural Networks, RAG, Transformers, LLMs,
Feature Engineering, Time-Series Analysis

1.2 Other Team Members

#	Name	Roll Number	Key Skills
2	Santosh C	250002021	Neural Networks, GANs, LLM, Transformers
3	Adwaith P	250003009	LLM, Transformer
4	Abhinav C	250003004	LLM, Transformer, Neural Networks
5	Sidhanth M	250005045	Web Development, JavaScript Frameworks
6	Manas Dhokar	250005025	LLM, Transformers

2. Problem Statement Preferences

Preference	Problem Statement	Domain
1	Personalized Mental Health Digital Twin using AI	AI / ML
2	Building Lightweight Multi-Task AI with MALORA	AI / ML
3	Pathway-Powered Agentic Gateway	AI / ML

3. Problem Statement

Most AI systems built around mental health operate on a simple classification premise: given some data about a person, decide whether they are depressed or not. This framing is clean and easy to evaluate, but it misrepresents how mental health actually works. Depression, burnout, and anxiety do not appear overnight as binary switches. They develop gradually, often over weeks or months, and they look different in every person.

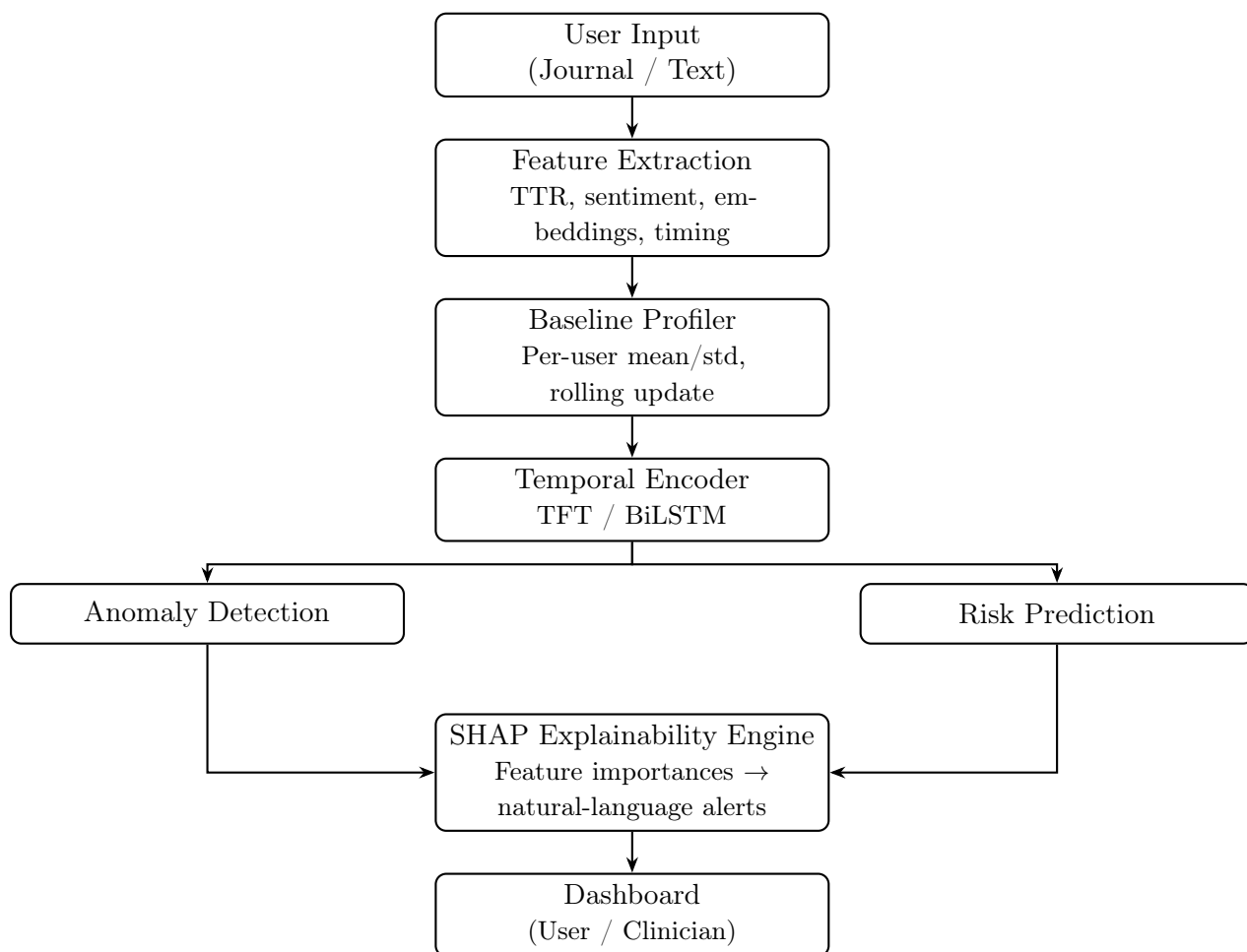
A model trained on population-level statistics learns what mental illness looks like on average — which is not particularly useful for catching it early in any specific individual.

One person’s tense writing is another’s natural style. These ambiguities make population-level thresholds unreliable for early detection, and they are precisely why current clinical screening tools like PHQ-9 are periodic and reactive rather than continuous and proactive.

What is actually needed is a system that learns the user first: their normal writing pace, their usual sentence structure, how their language shifts across moods and days. Only once that individual baseline is well-established can any deviation from it mean something. This is the core motivation behind the **Personalized Mental Health Digital Twin** — not to replace clinical judgment, but to offer a continuous, personalized layer of behavioral monitoring that can flag something worth paying attention to well before it becomes a crisis.

4. System Architecture

The full pipeline is shown below. Data flows from the user’s journal input through five sequential processing stages before surfacing on the dashboard.



The pipeline branches at the Temporal Encoder stage: anomaly detection and risk prediction run in parallel on the same latent trajectory. An anomaly detection algorithm

outputs an unexpected distance value; passing a raw distance metric into a risk model to forecast transitions like burnout is unreliable because a high anomaly score could reflect a singular, harmless outlier. The outputs are merged before the explainability engine generates human-readable summaries for the dashboard.

5. Dataset Plan

5.1 Primary Dataset — DAIC-WOZ

The Distress Analysis Interview Corpus (DAIC-WOZ) is the most suitable publicly available dataset for this work. It contains clinical interviews with participants across a range of PHQ-8 depression scores, with accompanying transcripts and self-reported mood labels that can serve as proxy supervision for the risk prediction module. The data is longitudinal enough to construct short multi-session trajectories per participant, which is exactly the temporal structure our model needs.

We supplement this with the CLPsych shared task datasets, which provide annotated social media text from Reddit and Twitter with mental health labels. These are useful for pre-training the feature extraction components, even though the domain differs from journaling.

5.2 The Cold-Start Problem and Synthetic Augmentation

Neither dataset provides what the system ultimately needs in production: dense, day-by-day journal entries from the same person over weeks. To handle this gap during development and testing, we will build a synthetic data generator that simulates realistic longitudinal journal sequences. The generator works by sampling a base behavioral profile (writing speed, average length, sentiment distribution) and then injecting controlled perturbations. This lets us test the anomaly detection and baseline modules against sequences where we know exactly when and how the deviation happens — which would be impossible with real data.

6. Module Descriptions

Module 1: Data Ingestion & Feature Extraction. Users submit daily text entries through a lightweight web interface. Each submission is timestamped and stored locally in encrypted form. From every entry we extract three layers of features: surface linguistic statistics (type-token ratio, average sentence length, VADER sentiment), semantic embeddings via `all-MiniLM-L6-v2`, and behavioral signals like entry length over time and submission regularity. Everything is normalized per user before being passed downstream.

Module 2: Personalized Baseline Profiler. After roughly ten entries, the system builds a per-user statistical profile: a rolling mean and standard deviation across all feature dimensions, updated via exponentially weighted moving average. Two windows are maintained — 7-day and 30-day — because comparing them lets us tell apart a rough week from a sustained trend. The baseline is intentionally simple: Gaussian statistics are transparent, auditable, and degrade smoothly when data is sparse.

Module 3: Temporal Mental State Encoder. The sequence of daily feature vectors forms a multivariate time series treated as the user’s mental state trajectory. A Temporal Fusion Transformer encodes this into a sequence of compact latent vectors — one per day — capturing temporal dependencies that point-in-time statistics would miss. If data

volume is insufficient for TFT, we fall back to a bidirectional LSTM. The latent trajectory is projected to 2D via UMAP for visualization on the dashboard, where deviations appear as direction changes in the user’s plotted path.

Module 4: Deviation & Anomaly Detection. Two parallel analytical tracks flag anomalies. Feature-level z -scores ($|z| > 2.0$) spot obvious independent baseline violations. For multidimensional shifts, we map structural deviations using the Mahalanobis Distance framework to account for dynamic feature cross-correlations:

$$D_M(\vec{x}) = \sqrt{(\vec{x} - \vec{\mu})^T \Sigma^{-1} (\vec{x} - \vec{\mu})}$$

We apply a Cumulative Sum (CUSUM) filter over these values. This allows the tracking architecture to mathematically decouple sudden, high-variance personal outliers (one bad day) from gradual, long-term toxic trajectory drift (burnout onset). Alerts are logged internally and only surfaced to the dashboard once thresholds clear for two consecutive days.

Module 5: Risk Prediction & Explainability Engine. An XGBoost classifier over a rolling window of anomaly signals and aggregated features produces a three-level risk estimate (*low / moderate / elevated*) for the coming week. XGBoost was chosen deliberately over a recurrent model because it is far easier to explain with SHAP values, and explainability is non-negotiable in anything touching clinical mental health. SHAP feature importances are mapped to templated natural-language summaries — grounded, auditable outputs rather than free-form generation.

7. Tech Stack

Component	Tool / Library	Reason
Feature extraction	SentenceTransformers, VADER, spaCy	Lightweight, runs fully local, no API calls needed
Baseline profiling	NumPy, pandas	Statistics at this layer should stay simple and auditable
Temporal encoding	PyTorch — TFT (pytorch-forecasting)	Built for multi-horizon time series; attention weights aid explainability
Fallback encoder	PyTorch — BiLSTM	Smaller footprint if TFT is overkill given data volume
Anomaly detection	scipy, scikit-learn	z -score and cosine distance; no reason to over-engineer this stage
Risk prediction	XGBoost	Interpretable by design; SHAP support is first-class
Explainability	SHAP	Produces per-feature attributions that map directly to user-facing text
Dashboard	React (Frontend) + FastAPI (Backend)	Decouples data processing from UI; enables rapid API iteration
Storage	SQLite (SQLCipher for encryption)	No external dependencies; user data stays on-device
Synthetic data	Custom Python generator	Controlled anomaly injection for module testing

8. Ethical Considerations & Privacy

- **No raw text stored or transmitted.** Journal entries are processed locally and immediately converted to feature vectors. The raw text is never written to any persistent storage and never leaves the user’s device. What is retained is the derived feature representation, not the words themselves.
- **On-device, encrypted processing.** All computation happens locally. The SQLite database is encrypted via SQLCipher. There is no cloud endpoint, no third-party data processor, and no network transmission in the prototype.
- **Explicit, granular consent.** Users opt in to each data modality separately. Onboarding explains in plain language what is being tracked, how it is used, and how to delete it.
- **The system does not diagnose.** Every risk output is framed as a change in behavioral patterns relative to the user’s own baseline — not a clinical assessment. Every screen that shows a risk signal includes a visible disclaimer that this is not a substitute for professional support. We are building a monitoring tool, not a diagnostic one.

9. Tentative Timeline

Phase	Dates	Key Deliverable
Setup & Research	May 22 – Jun 8	Literature review (DAIC-WOZ, CLPsych, digital biomarker)
Core Build	Jun 9 – Jun 25	Modules 1–4 built and individually tested on synthetic data
Mid-Evaluation	Jun 30	Working demo: user can input entries across simulated days
Advanced Features	Jul 1 – Jul 20	Module 5 complete (XGBoost risk model + SHAP summaries)
Finalization	Jul 21 – Aug 2	Final report (research paper style), GitHub cleanup and doc

10. Expected Outcomes & Extensions

The immediate goal is a working prototype — something a user can actually interact with over a period of days and get meaningful, personalized feedback from. Beyond the prototype:

- **Clean, reproducible codebase** with explicit module boundaries that make it straightforward for future contributors to swap in a better temporal encoder or a different anomaly detection method.
- **Final report written to conference paper standards**, documenting the system design, experimental results on DAIC-WOZ, and an honest account of limitations. The work is novel enough — particularly the dual-baseline design and the personalized thresholding approach — that a workshop paper submission is a realistic target.
- **A foundation for something larger.** Personalized behavioral monitoring at this level of granularity does not yet exist as an accessible, open-source tool. The modular architecture is designed with extension in mind: adding speech or wearable inputs, plugging in a reinforcement learning layer for adaptive interventions, or integrating with a mobile platform are all natural next steps that build directly on what we are constructing here.

This proposal is submitted for IITISoC 2026 under the
Science & Technology Council, IIT Indore.